Projeto POO 2026-1: Batalha Naval
Refatoração e Evolução

1. Contexto do trabalho

Você recebera um código Java funcional, porém monolítico, que implementa o jogo Batalha Naval em terminal. Esse código roda, mas foi escrito de um jeito propositalmente ruim: regras, UI, estado do jogo e lógica de CPU estão misturados.

Seu trabalho tem duas metas:

- Refatorar o código para um design orientado a objetos, preservando o comportamento.
- Evoluir o sistema com três implementações obrigatórias: validação de frota, melhoria da interface no terminal e persistência em banco de dados.

A ideia e transformar um sistema que funciona em um sistema bem estruturado, testável e fácil de manter.

2. O que você vai receber

- Um Main.java monolítico e jogável.
- Regras básicas do jogo já implementadas.
- Um comportamento que deve continuar existindo após a refatoração.

Você não vai começar do zero. Você vai refatorar e evoluir.

3. Regras do jogo base

- Tabuleiro 10x10.
- Coordenadas A-J e 1-10.
- Frota com 5 navios: tamanhos 5, 4, 3, 3, 2.
- Navios na horizontal ou vertical, sem sobreposição.
- Tiro retorna: água, acerto ou afundou.
- Vence quem afundar todos os navios do adversário.

4. Entregáveis

Você deve entregar:

1. Código refatorado com POO e funcionando.
2. Testes automatizados para o motor do jogo.
3. Persistência em banco com *schema* e dados gravados.
4. README.md explicando como rodar, testar e qualquer dependência.

O repositório deve permitir execução e testes de forma reproduzível.

5. Como conduzir o trabalho

Siga esta sequência. Ela reduz risco e facilita validação.

5.1 Etapa 1: estabilize o comportamento antes de mexer

5. Compile e execute o monolito original.
6. Rode partidas usando *seed* fixa para reproduzir resultados.
7. Identifique entradas e saídas típicas do programa.
8. Crie testes cobrindo regras centrais antes de refatorar. Refatorar sem testes aumenta muito o risco de quebrar comportamento.

6. Parte 1: Refatoração obrigatória

6.1 Objetivo

Separar o sistema em classes do domínio, serviços e UI, mantendo o jogo jogável em terminal.

6.2 Regras da refatoração

- O jogo deve continuar funcionando em terminal.
- O comportamento do motor deve ser preservado.
- UI não pode conter regra de domínio.
- Não vale manter tudo estático.
- Não vale depender de variáveis globais espalhadas para representar o estado.

6.3 Estrutura mínima esperada

Os nomes podem variar, mas as responsabilidades precisam existir. No mínimo:

- Jogo: controla fluxo, turnos e fim do jogo.
- Jogador como abstração de jogador, com implementações HumanPlayer e CpuPlayer.
- Tabuleiro: representa células, posicionamento e aplicação de tiros.
- Navio: tamanho, ocupação, hits e afundamento.
- Coordenada: *parsing* e validação de coordenadas.
- ResultadoTiro: água, acerto, afundou e detalhes associados.

6.4 Separação obrigatória entre UI e domínio

O domínio não pode imprimir nada. Isso inclui nada de System.out.println em Tabuleiro, Jogo, Navio, ValidadorDeFrota, etc. Também não pode existir Scanner dentro do domínio.

A UI em terminal deve apenas:

- mostrar menus e tabuleiros
- ler entradas
- chamar o domínio e serviços
- apresentar mensagens produzidas pelos resultados das operações

6.5 Testes obrigatórios

Você deve ter testes para:

- coordenadas validas e invalidas
- posicionamento de navios, incluindo colisão e limites
- aplicação de tiros: água e acerto
- detectar navio afundado
- fim de jogo quando a frota termina

Esses testes devem rodar sem terminal, sem Scanner e sem prints.

7. Parte 2: Implementações obrigatórias

7.1 Validação de frota

Implemente uma validação independente da UI. Ela deve checar:

- Frota correta: exatamente 5 navios com tamanhos 5, 4, 3, 3, 2.
- Dentro do tabuleiro: nenhuma célula fora do 10x10.
- Sem sobreposição: nenhuma célula compartilhada entre navios.
- Navios contínuos e retos: cada navio e uma linha reta horizontal ou vertical, sem buracos e sem diagonal.

Como implementar:

- Crie uma classe como ValidadorDeFrotas.
- Retorne um resultado com ok (boolean) e errors (lista de mensagens).
- A UI apenas mostra as mensagens e pede novo posicionamento quando necessário.

7.2 Melhoria de interface no terminal

Melhore a experiencia de uso no terminal, sem transformar o domínio em UI.

Requisitos mínimos:

- Dois tabuleiros lado a lado: seu tabuleiro e seus tiros.
- Cabeçalho A-J e linhas 1-10 alinhadas.
- Mensagens padronizadas para água, acerto, afundou e vitória.
- Mostrar quantos navios ainda estão vivos de cada lado.
- Menus com ações explicitas e opção de ajuda ou legenda.
- Posicionamento manual e automático, com mensagens claras quando rejeitar uma posição.

7.3 Persistência em banco de dados

Persista dados em um banco relacional. Use SQLite via JDBC.

O que gravar no mínimo:

- Partida: id, inicio, fim, vencedor, seed.
- Jogadores: id, nome, tipo (humano ou cpu), id da partida.
- Jogadas: id, id_partida, turno, jogador, coordenada, resultado.

Funcionalidades obrigatórias:

- Salvar automaticamente uma partida completa ao final, incluindo todas as jogadas.
- Listar histórico de partidas no menu inicial.
- Replay: escolher uma partida e reproduzir as jogadas no terminal, passo a passo ou avançando com Enter.

Arquitetura mínima para persistência:

- Crie uma camada própria para banco, por exemplo *Repository* ou *Dao*.
- Nada de SQL no domínio e na UI.
- Queries centralizadas.
- Criação de tabelas pode ser feita no startup do programa (auto-migrate).

8. Arquivo de configuração: game.properties

Junto com o código, você recebera um arquivo *game.properties*. Esse arquivo define parâmetros do jogo e da execução. A intenção e que o codigo leia a configuração no startup e se adapte sem alterações no fonte.

Durante a avaliação, o *game.properties* pode ser alterado para verificar se o seu projeto e parametrizável. Não fixe valores no código que deveriam vir da configuração.

8.1 Variáveis do arquivo

A tabela abaixo descreve as chaves esperadas.

Chave	**Descricao**
group.id, group.name	Identificacao do grupo para logs e persistencia.
project.version	Versao do projeto, util para rastrear mudancas.
board.size	Tamanho do tabuleiro (N para tabuleiro NxN).
board.columns.start, board.columns.end	Intervalo de letras para colunas na UI. Pode ser derivado de board.size.
fleet.sizes	Lista com tamanhos da frota. Ex: 5,4,3,3,2.
fleet.names	Nomes amigaveis dos navios, na mesma ordem de fleet.sizes.
fleet.adjacency_rule	Regra de encostar navios: NONE, ORTHO, ORTHO_DIAG.
rules.hit_grants_extra_shot	Se TRUE, acerto permite atirar novamente no mesmo turno.
rules.max_extra_shots	Limite de tiros extras quando hit_grants_extra_shot=TRUE.

cpu.strategy	Estrategia da CPU. Ex: RANDOM, HUNT, PARITY.
cpu.use_parity_preference	Se TRUE, CPU usa heuristica simples (padrao x+y par) quando aplicavel.
ui.show_own_ships	Se TRUE, mostra os navios do jogador no proprio tabuleiro.
ui.show_legend	Se TRUE, imprime legenda de simbolos na interface.
ui.replay_delay_ms	Delay do replay em milissegundos. 0 desliga atraso.
db.enabled	Liga ou desliga persistencia.
db.type	Tipo de banco. Para este projeto: sqlite.
db.sqlite.file	Caminho do arquivo .db do SQLite.
db.auto_migrate	Se TRUE, cria tabelas automaticamente no startup.
db.save_initial_fleet	Se TRUE, salva a frota inicial para replay completo.
game.seed	Seed fixa para reproducao. Se vazio, aleatorio.
game.mode	Modo de execucao: PLAY, LIST, REPLAY.

9. Requisitos técnicos

- Java 17 ou superior.
- Maven ou Gradle, ou instruções claras de compilação manual (utilize Spring Initializr https://start.spring.io/).
- SQLite via JDBC.
- Testes com JUnit.

10. Restrições

- Terminal apenas.
- Evite frameworks pesados para persistência. JDBC direto e o esperado.
- Não reescreva do zero. A entrega deve mostrar evolução incremental.

11. Checkpoints sugeridos

- Checkpoint 1: motor OO funcionando, com testes básicos, sem banco.
- Checkpoint 2: validação de frota integrada e interface melhorada.
- Checkpoint 3: banco persistindo partidas e replay funcionando.